

## 1. Disk Space & Log Rotation Management

The proposed layout uses LVM (Logical Volume Manager) so every functional area is a separate logical volume with its own size cap, combined with logrotate for automatic log lifecycle management. This isolates failure domains and gives an online-resizable safety margin.

- (root) — 8 GB: only OS binaries and configuration; kept small and stable so it rarely grows.
- var/log — separate 10 GB logical volume (XFS with project quota as a second safety net)
- home — 15 GB for user files, the most predictable, slow-growing segment.
- opt (applications) and /var/lib (app data) — 14 GB combined, sized for moderate growth with room to spare.
- 3 GB left as unallocated LVM free extents — a reserve pool that can be assigned via lvextend.
- logrotate configured with daily rotation, compression (gzip), a per-file maxsize guard, and a 10-rotation,

## Q2. Scheduler Optimization (Priority Scheduling)

The solution combines four complementary kernel-level levers rather than one single fix, since each targets a different source of context-switch overhead.

- Scheduler tuning: move latency-critical tasks to SCHED\_FIFO/SCHED\_RR (real-time policies) via chrt,
- CPU isolation and affinity: use isolcpus (or cpuset/cset shield) to dedicate specific cores exclusively to real-time tasks.
- Kernel modules: where a hot code path is dominated by syscall/interrupt overhead,
- Tickless kernel and preemption model: enable CONFIG\_NO\_HZ\_FULL so idle/isolated cores stop taking periodic timer interrupts

## Q3. Fast Processing of 1 Million Log Entries

The design splits the file into independent chunks, processes them in parallel with fast streaming tools, and merges results using an algorithm that avoids re-sorting the entire dataset.

- Split the file into 8 (or  $N$  = number of CPU cores) roughly equal chunks using `split -n l/8`, avoiding cutting lines mid-record.
- Process chunks in parallel with `xargs -P8` (or GNU parallel),
- Each chunk is pre-sorted where ordering/grouping is needed,
- `uniq -c` or an awk associative array aggregates counts/summaries from the merged stream.
- Final output is written once with buffered I/O rather than many small writes, minimizing disk thrashing.

#### Q4. Identifying & Controlling a Faulty High-CPU Process

The strategy follows a four-stage pipeline: detect, diagnose, act (with three graduated options), and safeguard/monitor — escalating only as far as necessary.

- Detect & identify: `top/htop` or `ps aux --sort=-%cpu` to find the exact PID consuming 90% CPU.
- Diagnose root cause: `strace -p <PID>` to see what syscalls it's stuck in,
- Option A — Throttle (least disruptive): `renice +19 <PID>` to lower scheduling priority,
- Option B — Isolate: `taskset` to move the faulty process onto specific cores away from the ones serving critical services,
- Option C — Controlled restart: if the process is unresponsive,

#### Q5. VM Resource Allocation on a 16 GB Host

The proposed strategy starts from the static baseline (4 GB host + 2 GB x 5 VMs = 14 GB, 2 GB reserved) and layers three virtualization memory-management techniques on top so the 2 GB headroom is shared efficiently rather than wasted.

assigning it to any single VM.

- Memory ballooning: the hypervisor's balloon driver reclaims idle RAM from lightly loaded VMs
- KSM (Kernel Samepage Merging): since the 5 VMs likely run similar OS images,
- Controlled overcommit with a swap/zram safety net: the hypervisor may allow soft overcommitment above the 2 GB/VM baseline during bursts,